UNIVERSITE IBN ZOHR
FACULTE DES SCIENCES
AGADIR

جامعة ابن زهر
كليـــــة العلـــــوم
أكاديـر

Rapport du projet de systèmes distribués sous le thème :

Conception et mise en œuvre d'un système distribué pour prédire et détecter les maladies des cultures utilisant l'apprentissage profond

Préparé par :

- Houcine Aberhache
- Aimane Achibane
- Abdessamad Jamed
- Salah Eddine Saâdaoui
- Mohammed Benkharraz
- Elmahdi Bahadia

Encadré par :

- Mr. El Habouz

Année universitaire 2025-2026

Table de matière

Chapitre 1 : Présentation du projet et architecture du système
I. Objectif du projet....06
II. Technologies utilisées....06
III. Contexte....07
IV. Architecture Logicielle....08
IV.1. Description générale de l'architecture du système....08
IV.1.1. Architecture distribuée de système....08
IV.1.2. Fonctionnement général....08
IV.1.3. Type de communication utilisé....08
IV.2. Déploiement distribué....09
IV.2.1. Séparation des serveurs....09
IV.2.2. Topologie du réseau....10
IV.2.3. Gestion des appels distants....10
IV.2.4. Scalabilité....11
IV.2.5. Sécurité....11
IV.2.6. Avantages du déploiement distribué....11
Chapitre 2 : Analyse Et Conception
I.Cahier de charges....12
I.1. Problématique....12
I.2. Solution....12
I.3. Objectifs....12
I.4. Identification des besoins....12
I.4.1. Besoins fonctionnels....13
I.4.2. Besoins non fonctionnels....13
II. Analyse Et Conception UML....13
II.1. Définition....13
II.2. Conception UML....14
II.2.1. Outils utilisé....14
II.2.2. Diagramme de cas d'utilisation....14
II.2.3. Diagramme de classe....16
II.2.4. Diagramme de séquence....18
II.2.4.1. Diagramme de séquence "Authentification"....18
II.2.4.2. Diagramme de séquence "Gérer les clients"....19
II.2.4.3. Diagramme de séquence "Effectuer une opération"....20
II.2.5. Diagramme de déploiement....21
Chapitre 3 : Réalisation et Implémentation
I.Modèle MVT "Model,View,Template"....23
II. Interfaces....24
II.1. Espace Client....24
II.2. Espace Administrateur....27
Conclusion et prespectives....28
Webliographie....29

Liste des figures

Figure 1: Logo GitHub 09
Figure 2: Logo UML 14
Figure 3: Logo StarUML 14
Figure 4: Diagramme de cas d'utiisation d'un Administrateur 15
Figure 5: Diagramme de cas d'utiisation d'un Client 15
Figure 6: Diagramme de classe 16
Figure 7: Diagramme de séquence "Authentification" 18
Figure 8: Diagramme de séquence "Gestion des clients" 19
Figure 9: Diagramme de séquence "Effectuer une opération" 20
Figure 10: Diagramme de déploiement 21
Figure 11: Diagramme de modèle MVT 23
Figure 12: Interface "Page d'accueil" 24
Figure 13: Interface "Login" Pour les clients 24
Figure 14: Interface "Créer un copmte" Pour les clients 25
Figure 15: Interface "Dashboard" 25
Figure 16: Interface "Résultats dune opération" 26
Figure 17: Interface "Consulter l'historique" 26
Figure 18: Interface "Login" Pour l'admin 27
Figure 19: Interface "Dashboard" Pour l'admin 27

Liste des abréviations

Abréviation	*Désignation*
UML	Unified Modeling Language
VSC	Visual Studio Code
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
SGBD	Système de Gestion de Base de Données
CNN	Convolutional Neural Network
gRPC	Google Remote Procedure Call
SQL	Structured Query Language
MVT	Model, View, Template
DCU	Diagramme de cas d'utilisation
DC	Diagramme de classes
HTTPS	HyperText Transfer Protocol Secure

Introduction Général

Dans le cadre du module des systèmes distribués du Master d'Excellence (ADIA), ce projet a pour objectif la conception et la réalisation d'un système distribué basé sur des technologies de communication à distance telles que RPC, RMI ou CORBA. L'application développée permet de répartir les différentes fonctionnalités du système sur plusieurs machines ou serveurs, tout en assurant une communication efficace entre les composants.

L'intérêt des systèmes distribués réside dans leur capacité à améliorer la performance, la flexibilité et la scalabilité des applications modernes. Cependant, leur mise en œuvre pose plusieurs défis, notamment la gestion des appels distants, la synchronisation des composants, la tolérance aux pannes et la fiabilité des échanges entre client et serveur. Ce projet vise ainsi à mettre en pratique les notions théoriques vues en cours à travers une application concrète intégrant une architecture distribuée et une interface web accessible via un navigateur.

Pour la réalisation de ce projet, différentes technologies ont été utilisées selon les besoins, notamment des langages comme Python, ainsi que des frameworks web tels que Django. L'utilisation de GitHub a également permis d'assurer le suivi des versions et la collaboration entre les membres de l'équipe.

Ce rapport présente l'ensemble des étapes suivies pour la réalisation du projet, depuis la phase de conception et de planification jusqu'à l'implémentation, les tests et le déploiement. Il décrit l'architecture du système, les choix techniques effectués, ainsi que les diagrammes UML permettant de mieux comprendre le fonctionnement global de l'application distribuée.

Présentation du projet et architecture du système

I. Objectif du projet :

Ce projet consiste à concevoir et développer un système distribué pour la détection et la prédiction des maladies des cultures, basé sur un modèle CNN (Convolutional Neural Network) entraîné via l'apprentissage profond. Les utilisateurs peuvent uploader des images de leurs plantes via une interface web, et le système analyse l'image pour :

- Prédire la maladie,
- Détecter les zones potentiellement affectées,
- Proposer une solution ou un traitement adapté.

II. Technologies utilisées :

Le projet utilise un ensemble cohérent de technologies permettant de construire un système distribué performant pour la détection des maladies des cultures :

- **gRPC** : utilisé pour les communications distantes rapides entre le serveur web (Django) et le serveur d'inférence contenant le modèle CNN. gRPC permet un échange efficace d'images et de données grâce au protocole HTTP/2 et à la sérialisation Protobuf.
- **Modèle CNN** : un réseau de neurones convolutionnel spécialisé dans l'analyse d'images, utilisé pour prédire la maladie, détecter les zones affectées et fournir une recommandation de traitement.
- **Back-end "Django" :** framework web robuste assurant la gestion des requêtes utilisateurs, l'upload des images, la communication avec le serveur gRPC, et la préparation des résultats pour le front-end.
- **Front-end "Django et HTML, CSS et Bootstrap" :** technologies web classiques utilisées pour construire une interface utilisateur simple, interactive et accessible permettant l'upload d'images et l'affichage des résultats.

III. Contexte :

L'utilisation d'un système distribué s'est imposée comme une nécessité pour plusieurs raisons liées à la nature du projet et aux exigences du traitement d'images par apprentissage profond.

- Pourquoi un système distribué est nécessaire :

  - **Séparation des charges de travail** : L'analyse d'images par un modèle CNN est une tâche coûteuse en ressources. En séparant le serveur web (Django) du serveur d'inférence (gRPC + CNN), on évite la surcharge et on garantit une meilleure réactivité de l'interface utilisateur.
  - **Performances optimisées** : Le serveur d'IA peut être déployé sur une machine plus puissante (CPU/GPU), tandis que le serveur web peut rester léger et scalable.
  - **Flexibilité du déploiement** : Chaque composant peut évoluer indépendamment (mise à jour du modèle, optimisation du backend, changement d'hébergement…).
  - **Communication rapide** : gRPC offre une solution idéale pour relier plusieurs machines via un canal hautes performances.

- Défis rencontrés :

  - **Gestion des communications entre plusieurs machines** : Assurer une communication fiable et rapide entre Django et le serveur gRPC, notamment pour transmettre des images.
  - **Scalabilité du système** : Permettre au système de gérer plusieurs utilisateurs simultanés tout en maintenant des temps de réponse courts.
  - **Synchronisation et cohérence** : Garantir que les échanges entre le serveur web et le serveur d'IA se déroulent sans perte de données et avec une gestion correcte des requêtes.
  - **Gestion des erreurs réseau** : Anticiper les timeouts, interruptions de connexion, mauvaises sérialisations ou erreurs durant l'inférence.
  - **Séparation des environnements** : Déployer l'IA sur une machine dédiée tout en laissant le web accessible au public implique une bonne gestion des accès et de la sécurité.

IV. Architecture Logicielle :

IV.1. Description générale de l'architecture du système :

IV.1.1. Le système repose sur une architecture distribuée comprenant :

- **Client Web :** Interface permettant d'uploader une image.
- **Serveur Web (Backend) :** Réceptionne les images, prépare la requête gRPC.
- **Serveur d'IA gRPC :** Contient le modèle CNN chargé d'effectuer la prédiction et la détection.
- **Base de données :** Peut stocker l'historique des uploads, résultats et les informations des utilisateurs.

IV.1.2. Le fonctionnement général :

1. L'utilisateur va créer un compte et puis s'authentifier
2. L'utilisateur envoie une image via l'interface web.
3. Le backend l'envoie au serveur IA via gRPC.
4. Le modèle CNN analyse l'image et renvoie :
    - La maladie prédite,
    - Le pourcentage de confidence,
    - La zone affectée,
    - Une recommandation ou solution.
5. L'interface affiche le résultat à l'utilisateur dans une Dashboard.

IV.1.3. Type de communication utilisé

- **Rapidité et Efficacité :** gRPC assure une interaction rapide, efficace et fiable.
- **Protocole Optimisé :** Il s'appuie sur HTTP/2 pour une transmission optimisée des données, y compris des images volumineuses.
- **Sérialisation Compacte :** L'utilisation de Protocol Buffers (Protobuf) réduit la taille des messages échangés, améliorant ainsi la vitesse.
- **Performance Supérieure à REST/JSON :** Contrairement aux approches REST basées sur JSON, gRPC permet un échange binaire compact, offrant une latence plus faible et un débit plus élevé, ce qui est idéal pour le transfert rapide de données d'images et de réponses d'IA.
- **Développement Simplifié :** gRPC facilite la génération automatique des interfaces client/serveur, garantissant une communication fortement typée et réduisant les risques d'erreurs.

- **Gestion du code source et collaboration (GitHub) :**
    - Plateforme de gestion du projet : GitHub est utilisé comme système de contrôle de version pour assurer le suivi, la centralisation et la sécurisation du code source.
    - Organisation par branches : Chaque branche GitHub correspond à une partie spécifique du projet (par exemple : traitement des images, modèle de détection, backend gRPC, interface utilisateur …), permettant un développement modulaire et parallèle.
    - Travail collaboratif : Cette organisation facilite la collaboration entre les membres de l'équipe, en évitant les conflits de code et en permettant des tests indépendants avant l'intégration finale.
    - Intégration et stabilité : Les fonctionnalités développées sur chaque branche sont fusionnées progressivement dans la branche principale (main) après validation, garantissant la stabilité et la qualité du projet.



Figure 1 : Logo GitHub

IV.2.Déploiement distribué

Le système est déployé selon une architecture distribuée dans laquelle chaque composant remplit un rôle spécifique sur des machines potentiellement distinctes. L'objectif principal de ce déploiement est de garantir la performance, la scalabilité et la fiabilité du système tout en optimisant les ressources matérielles.

IV.2.1.Séparation des serveurs

Le projet utilise une séparation claire entre plusieurs services :

- **Serveur Web (Django)** : héberge l'interface utilisateur, gère les uploads d'images, les sessions utilisateurs et la logique d'application. Il ne réalise aucun calcul intensif, ce qui permet d'assurer une réponse rapide à l'utilisateur.

- **Serveur d'inférence IA (gRPC + CNN)** : exécute le modèle CNN, souvent sur une machine dédiée disposant de GPU/CPU puissants. Ce serveur est isolé afin de garantir que les calculs lourds n'affectent pas les performances de la plateforme web.

- **Base de données (SQLite + Cloud Storage) :** Notre système utilise SQLite comme base de données principale, tandis qu'un **service Cloud** sert à stocker les images ou fichiers volumineux.

L'architecture adopte alors une logique hybride :

- Identifiants des images,
- Les informations des utilisateurs,
- Date d'envoi,
- Résultat de la classification,
- Leurs logs,
- Performances du modèle.

Cette séparation permet une meilleure gestion des ressources : le serveur web peut être répliqué facilement pour gérer une forte charge, tandis que le serveur IA peut être optimisé pour le calcul intensif.

IV.2.2. Topologie du réseau :

La communication entre les composants suit une architecture orientée services :

1. L'utilisateur interagit uniquement avec le serveur web.
2. Le serveur web envoie les images au serveur IA via gRPC.
3. Le serveur IA retourne les résultats (maladie détectée, probabilité, zone affectée, solution).
4. La réponse est affichée à l'utilisateur.

Cette communication se déroule sur un réseau interne sécurisé ou via un tunnel sécurisé lorsque les serveurs sont hébergés dans des environnements différents.

IV.2.3. Gestion des appels distants

- Le serveur Django agit comme **client gRPC**: il appelle le service d'inférence via les stubs générés à partir du fichier .proto.
- Le serveur IA agit comme **serveur gRPC**, exposant des méthodes distantes pour le traitement d'images.
- Les messages échangés utilisent **Protocol Buffers**, garantissant rapidité et efficacité.
- Une gestion des timeouts, reconnexions et exceptions est intégrée pour assurer la robustesse du système.

IV.2.4. Scalabilité :

Le déploiement distribué permet d'évoluer facilement :

- Le serveur IA peut être dupliqué (plusieurs instances d'inférence) pour traiter plusieurs images simultanément.
- Un répartiteur de charge (load balancer) peut être ajouté devant Django ou devant les serveurs IA.
- Le système peut migrer vers le cloud (AWS, GCP, Azure) sans changement majeur dans l'architecture.

IV.2.5. Sécurité :

- Les communications entre Django et le serveur IA peuvent être chiffrées.
- Les images utilisateur sont validées et nettoyées avant traitement.
- Le serveur IA peut être isolé dans un réseau privé, inaccessible depuis Internet.

IV.2.6. Avantages du déploiement distribué :

- Haute performance grâce à la separation du calcul IA.
- Tolérance aux pannes : la chute d'un composant ne bloque pas le système.
- Possibilité d'optimiser chaque machine selon son rôle.
- Mise à l'échelle simplifiée selon la demande.

❖ Conclusion

Ce chapitre a permis de présenter les objectifs du projet, les technologies utilisées ainsi que l'architecture globale du système distribué proposé. Le choix d'une architecture basée sur Django, gRPC et un modèle CNN permet de séparer efficacement les responsabilités, d'optimiser les performances et d'assurer la scalabilité du système.

Cette organisation constitue une base solide pour la mise en œuvre, le déploiement et l'évolution future de l'application de détection et de prédiction des maladies des cultures.

Chapitre 2 : Analyse Et Conception

I. Cahier de charges

I.1. Problématique

La gestion agricole reste traditionnelle, entraînant une perte de temps, un suivi inefficace, des informations dispersées et un manque de recommandations adaptées, ce qui complique la prise de décision et l'identification des maladies des plantes.

I.2. Solution

Mettre en place une application web distribuée pour centraliser les informations, assister les agriculteurs dans la prise de décision, détecter les maladies et rendre la gestion plus rapide, précise et accessible.

I.3. Objectifs

Afin d'atteindre les objectifs, nous allons concevoir et réaliser une application web qui répond et garantira aux fonctionnalités suivantes :

- Détection automatique des maladies des plantes à partir d'images ;
- Accès instantané à des recommandations de traitement ;
- Centralisation des informations agricoles utiles dans une seule plateforme ;
- Interaction simple, ergonomique et rapide pour les utilisateurs ;
- Système scalable et évolutif grâce à l'utilisation de microservices.

I.4. Identification des besoins

L'identification des besoins dans un cahier des charges est une étape cruciale dans la planification et le développement de tout projet. On distingue deux types des besoins :

- Besoins fonctionnels;
- Besoins non fonctionnels.

I.4.1. Besoins fonctionnels

Le système doit inclure :

- Authentification sécurisée ;
- Gestion des projets (création, upload d'images, consultation de l'historique) ;
- Gestion des clients et sous-admins ;
- Gestion des dashboards (ajout, modification, suppression).

I.4.2. Besoins non fonctionnels

Pour notre projet, l'application devra répondre aux exigences suivantes :

- Ergonomie et facilité d'utilisation ;
- Sécurité et confidentialité des données ;
- Fiabilité et cohérence des opérations ;
- Compatibilité et portabilité sur toutes les plateformes ;
- Maintenance et réutilisabilité du code ;
- Performance rapide et faible latence ;
- Flexibilité et extensibilité pour de futures évolutions ;
- Cohérence et intégrité des données.

II. Analyse Et Conception UML

Dans le cycle de vie d'un projet, la conception est essentielle pour garantir la qualité de l'application. Ce chapitre présente l'analyse et la modélisation du système avec UML, traduisant les besoins en diagrammes de cas d'utilisation, de classe, de séquence et de déploiement.

II.1. Définition

L'Analyse et Conception avec UML (Unified Modeling Language) est une méthode de modélisation permettant de représenter graphiquement les aspects d'un système logiciel. UML est utilisé tout au long du développement, de l'analyse des besoins à la conception et à la documentation.

- **Analyse UML :** Cette phase permet de comprendre les besoins du client et de définir les fonctionnalités du système, notamment à travers les diagrammes de cas d'utilisation qui identifient les acteurs et leurs interactions.
- **Conception UML :** Cette phase décrit la structure interne du système et les interactions entre ses composants, avec des diagrammes comme les diagrammes de classes et de séquence pour représenter l'architecture logicielle.

II.2. Conception UML

UML utilise des diagrammes pour représenter la structure et le comportement d'un système, facilitant ainsi la compréhension des systèmes complexes de manière claire et graphique, sans passer par le code. Il est principalement utilisé dans le développement logiciel, mais aussi dans d'autres domaines comme l'industrie et l'ingénierie pour modéliser des processus.



Figure 2 : Logo UML

II.2.1. Outil utilisé

StarUML : Est un logiciel de modélisation UML permettant de concevoir et visualiser divers diagrammes, comme les diagrammes de classes, de séquence, d'activité et de cas d'utilisation. Il aide à planifier, concevoir et documenter des systèmes logiciels grâce à des modèles visuels et offre des fonctionnalités comme la génération de code, la rétro-ingénierie et la prise en charge de plusieurs types de diagrammes.



Figure 3 : Logo du StarUML

II.2.2. Diagramme de cas d'utilisation

- **Notion :**

En UML, le diagramme de cas d'utilisation modélise le comportement global d'un système et capture ses exigences fonctionnelles. Il décrit les principales fonctionnalités et les interactions avec les acteurs, montrant ce que fait le système et comment il est utilisé, sans détailler son fonctionnement interne, ce qui en fait un outil essentiel pour la conception.

➢ Diagramme de cas d'utilisation pour un administrateur :

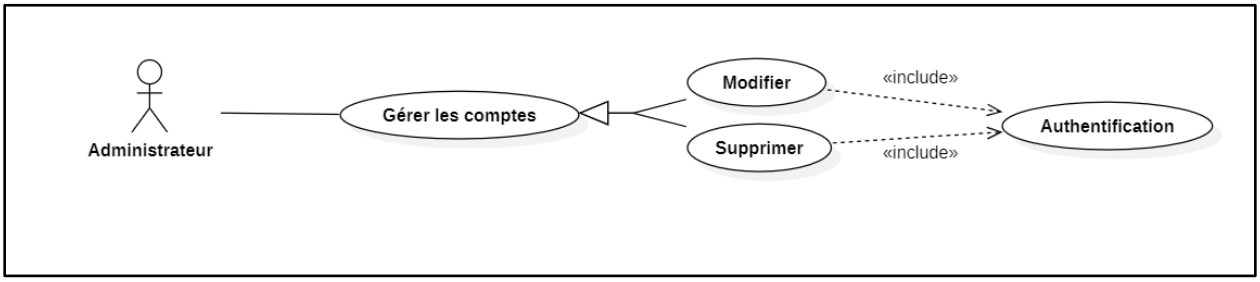


Figure 4 : Diagramme de cas d'utilisation d'un administrateur

Comme étant un utilisateur de l'application l'administrateur intervient par plusieurs cas d'utilisation qu'on explique brièvement :

- **Gérer les comptes :** Permet à l'administrateur de gérer les informations et les comptes des clients, et de les supprimer si nécessaire.

➢ Diagramme de cas d'utilisation pour un client :

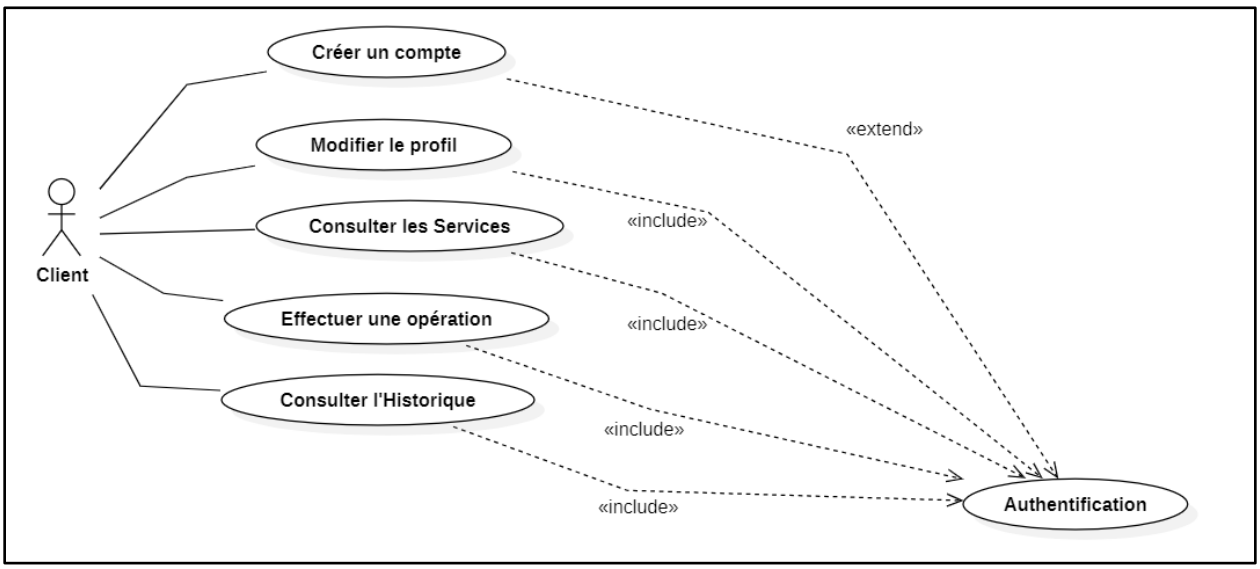


Figure 5 : Diagramme de cas d'utilisation d'un client

Et en ce qui concerne l'administrateur, il peut effectuer les actions suivantes :

- **Créer un compte.**
- **Modifier son profil.**
- **Consulter les services.**
- **Effectuer une opération**.
- **Consulter l'historique**.

II.2.3. Diagramme de classe

Le diagramme de classes UML est un diagramme structurel qui modélise la structure statique d'un système. Il représente les classes, leurs données, opérations et relations, en se concentrant sur leur logique générale sans détailler l'implémentation interne.

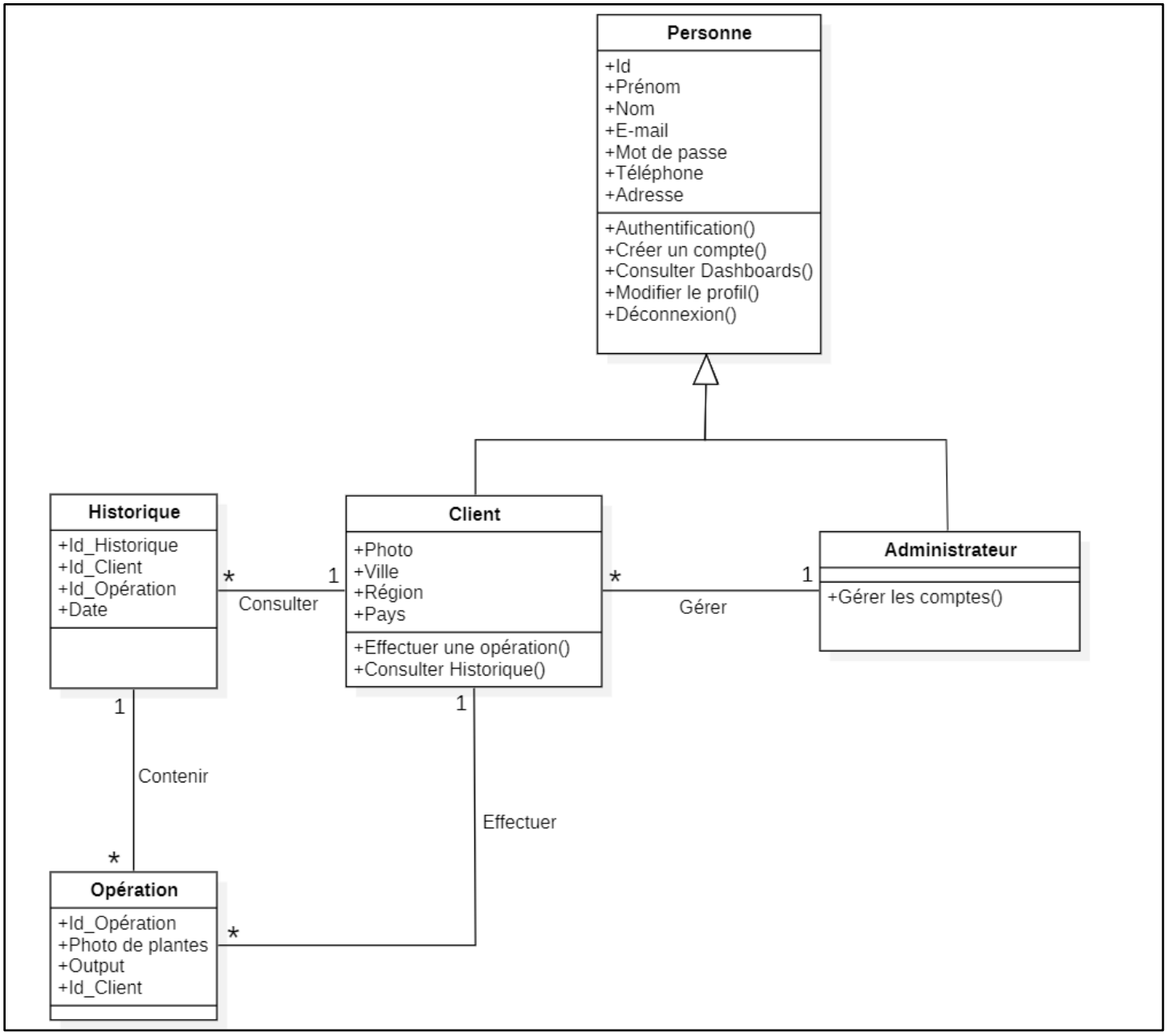


Figure 6 : Diagramme de classe

Ce diagramme de classe illustre les relations entre les différents rôles dans notre système, notamment l'administrateur et les clients.

Voici une description détaillée de chaque classe et des relations entre elles :

❖ Classes Principales

➢ Personne:

- **Attributs :** Id_Personne, Prénom, Nom, E-mail, Mot de passe.
- **Relation :** Personne est une classe parent pour Client et Administrateur.

➢ Client (hérite de Personne)

- **Méthodes :** Effectuer une opération() / Consulter l'historique().
- **Relation :** Un Client peut effectuer une ou plusieurs opérations et consulter les dernières analyses effectuées, avec leurs dates.

➢ Administrateur (hérite de Personne)

- **Méthodes :** Gérer les comptes().
- **Relation :** Un Administrateur gère plusieurs aspects du système.

➢ Opération

- **Attributs :** Id_Opération, Id_Client, Titre, Photo de plantes, Discreption, Output.
- **Relation :** Un Client peut effectuer une ou plusieurs opérations.

➢ Historique

- **Attributs :** Id_Historique, Id_Opération, Id_Client, Date.
- **Relation :** Un Historique est concerné par un seul client et contient plusieurs opérations.

❖ Relations Entre les classes

- **Administrateur – Client :** Un administrateur peut gérer un ou plusieurs clients.
- **Client - Opération :** Un client peut effectuer une ou plusieurs opérations.
- **Historique - Client :** Un historique concerne un seul client, et un client peut consulter plusieurs historiques.
- **Historique - Opération :** Un historique peut contenir plusieurs opérations.

II.2.4. Diagramme de séquence:

Le diagramme de séquence est un diagramme UML qui représente les interactions entre les objets d'un système dans un ordre temporel précis. Il montre les échanges de messages dans un scénario et permet de détailler les cas d'utilisation du système.

II.2.4.1. Diagramme de séquence « Authentification »

L'authentification vérifie les informations d'un utilisateur pour garantir la confidentialité des données. Si les données sont complètes et correctes, l'accès est accordé ; sinon, un message d'erreur s'affiche et la page de connexion est réaffichée.

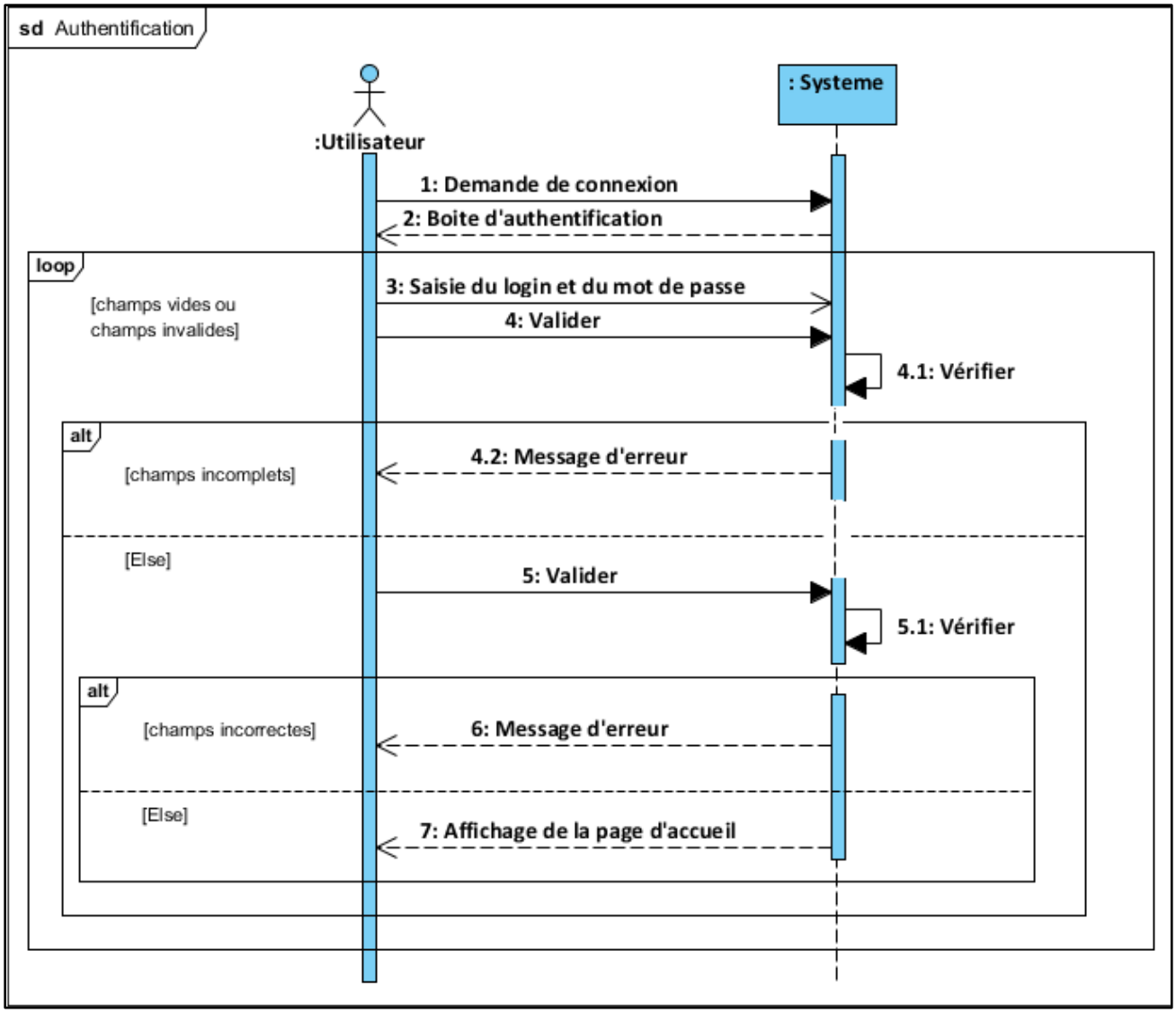
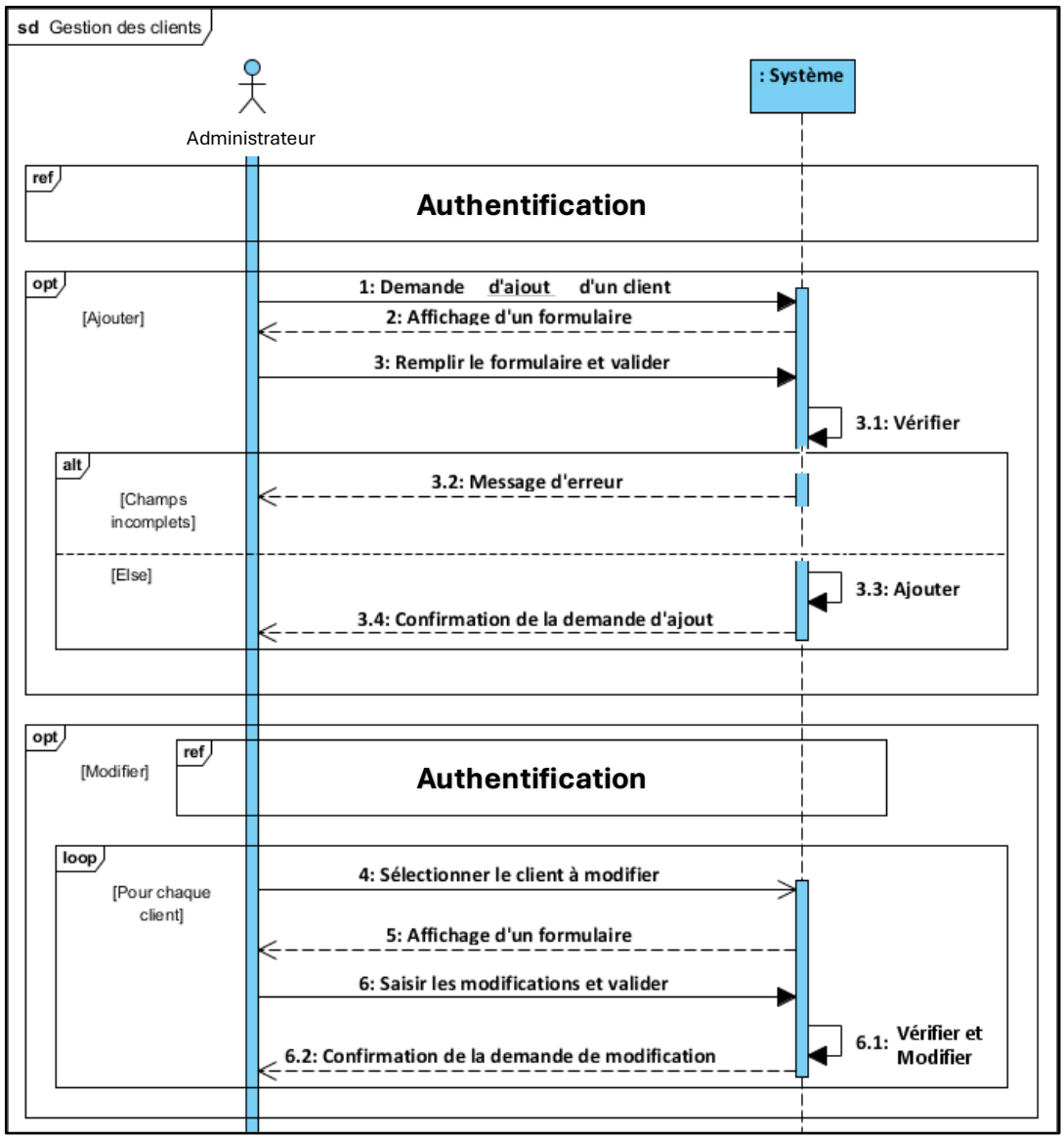


Figure 7 : Diagramme de séquence "Authentification"

II.2.4.2. Diagramme de séquence « Gérer les clients »

Après l'authentification, l'administrateur effectue une demande de gestion des clients. Trois scénarios sont représentés chacun d'entre eux correspond à un choix, d'où l'utilisation du fragment de type "Opt" indiquant que ces scénarios arrivent dans n'importe quel ordre.

- **Ajouter un client :** Après l'affichage du formulaire, l'utilisateur saisi les informations d'un client et valide l'action.
- **Modifier un client :** L'administrateur sélectionne le client concerné, saisit les modifications dans un formulaire et valide l'opération.
- **Supprimer un client :** l'administrateur sélectionne le client concerné et valide la suppression.



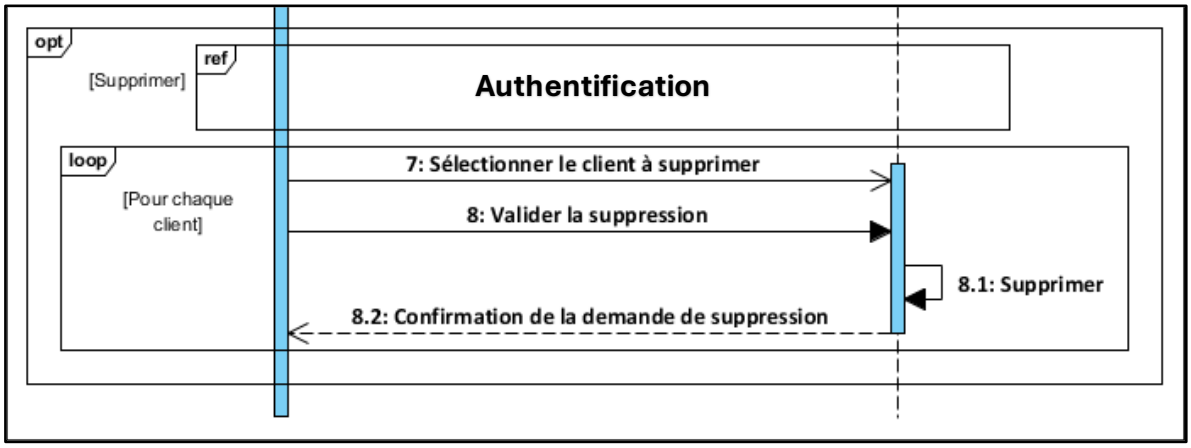


Figure 8 : Diagramme de séquence "Gestion des clients"

II.2.4.3. Diagramme de séquence « Effectuer une opération »

Après l'authentification, l'utilisateur peut effectuer une opération.

- **Uploader une image** : Après la création du projet, l'utilisateur peut effectuer une opération.

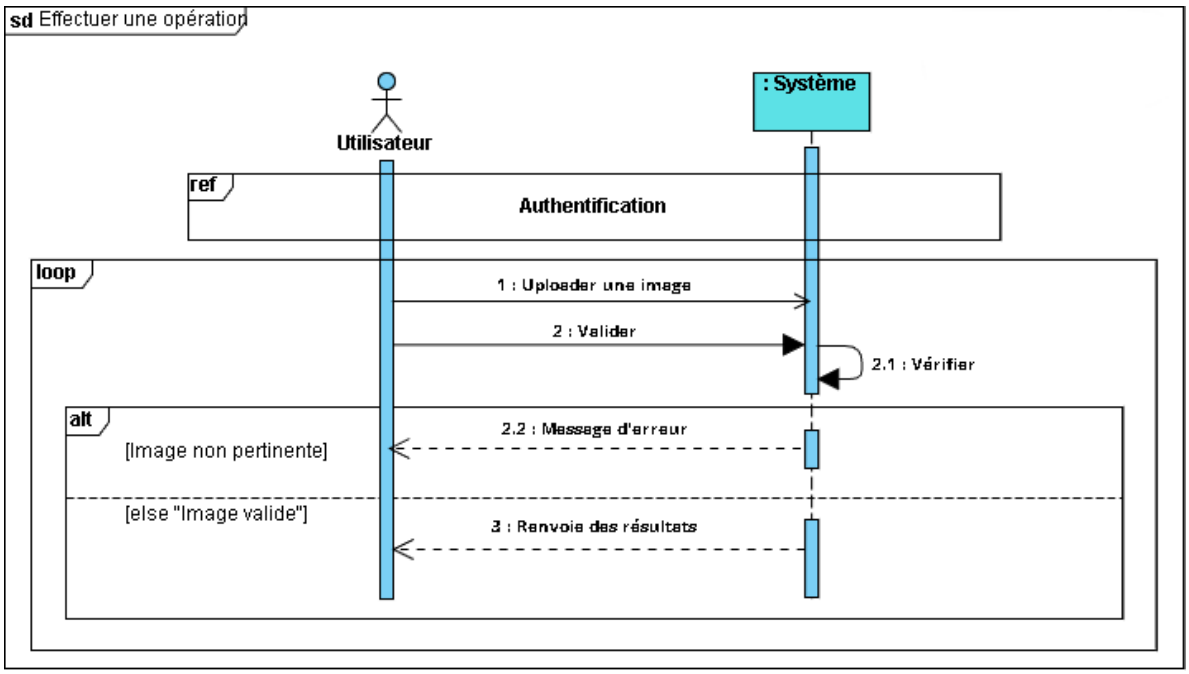


Figure 9 : Diagramme de séquence "Effectuer une opération"

II.2.5. Le diagramme de déploiement

Le diagramme de déploiement est un diagramme UML qui représente l'architecture physique du système. Il montre la répartition de l'application sur les machines ou serveurs et le déploiement des composants logiciels, ainsi que les relations entre clients, serveurs, bases de données et services, permettant de comprendre l'environnement d'exécution et l'organisation globale du système.

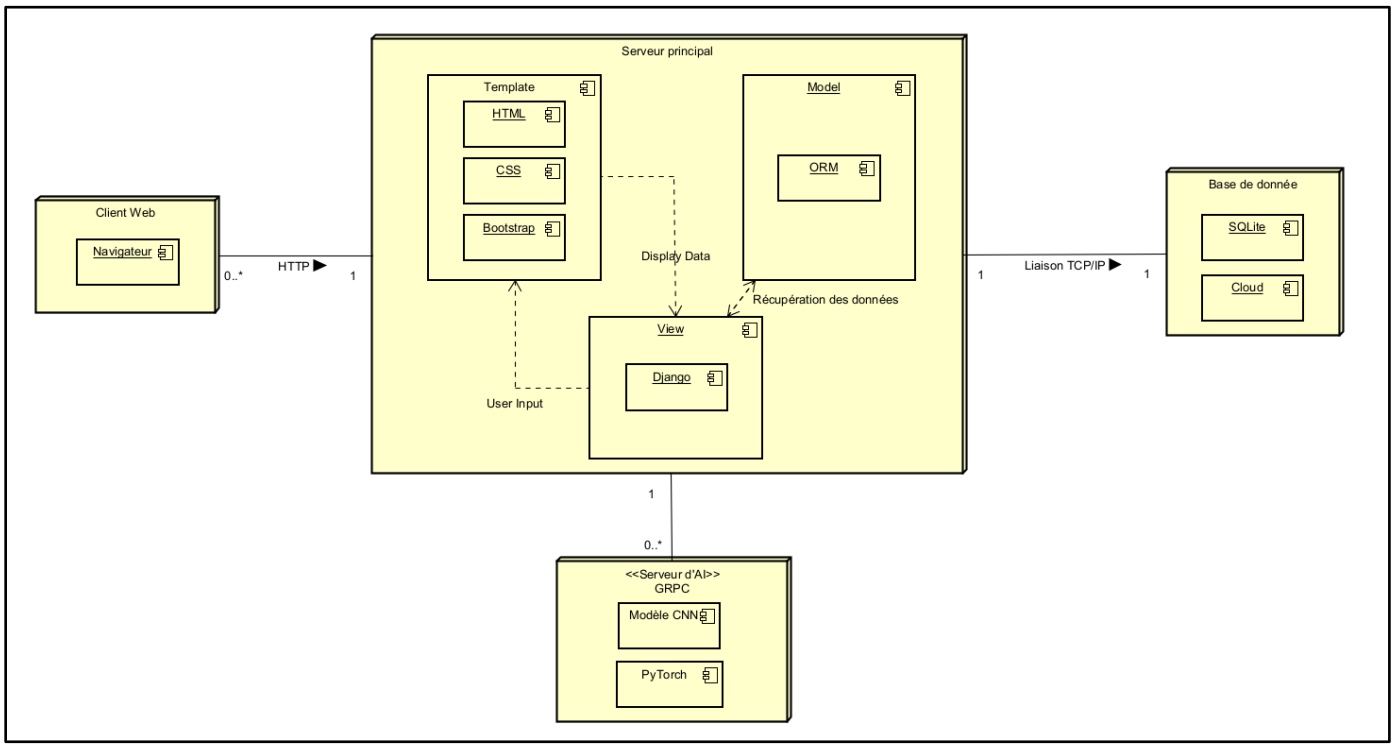


Figure 10 : Diagramme de déploiement

❖ **Client Web**

- L'utilisateur accède à l'application via un navigateur web.
- Les requêtes sont envoyées au serveur principal en utilisant le protocole HTTP.

❖ **Serveur principal**

- Héberge l'application web et la logique principale du système.
- La partie interface est gérée par des templates HTML, CSS et Bootstrap pour l'affichage des pages.
- La couche View (Django) traite les requêtes de l'utilisateur, gère les interactions et l'affichage des données.
- La couche Model utilise un ORM pour communiquer avec la base de données et récupérer les informations nécessaires.

❖ **Base de données**

- Stocke les informations de l'application.
- Utilise SQLite, avec possibilité d'hébergement local ou dans le cloud.
- La communication avec le serveur principal se fait via une liaison TCP/IP.

❖ **Serveur d'Intelligence Artificielle**

- Serveur dédié au traitement et à l'analyse des images.
- Contient un modèle CNN implémenté avec PyTorch.
- La communication avec le serveur principal se fait via le protocole gRPC.

❖ **Interaction globale**

- Le client envoie une requête au serveur principal via HTTP.
- Le serveur principal traite la requête et récupère les données nécessaires depuis la base de données.
- S'il y a une demande de prédiction, il communique avec le serveur d'IA via gRPC.
- Les résultats sont ensuite renvoyés à la vue Django et affichés à l'utilisateur via l'interface web.

❖ Conclusion

La phase conceptuelle est une étape fondamentale pour la réalisation de n'importe quel projet. Elle permet de faciliter la conception du système d'information ainsi que l'implémentation de la base de données et du traitement. Dans ce chapitre, nous avons présenté les différentes étapes essentielles du système élaboré pour l'analyse et la conception de notre application, en suivant le processus de normalisation UML et les différents diagrammes afin de faciliter la phase de réalisation. En premier lieu, nous avons présenté le diagramme de classes, ensuite nous avons recensé les cas d'utilisation associés aux différents acteurs ainsi que les diagrammes de séquence qui correspondent à ces cas d'utilisation. Enfin, nous avons présenté le diagramme de déploiement, qui permet de décrire l'architecture physique du système et la répartition des composants logiciels sur les différents serveurs.

Dans le chapitre suivant, nous allons chercher les moyens et les outils possibles pour développer cette application. Nous présenterons l'implémentation de notre application, les outils et les environnements utilisés. Des copies d'écran seront ajoutées pour montrer les fonctionnalités de notre application.

Chapitre 3: **Réalisation et implémentation**

❖ Intoduction

Le développement d'un projet informatique nécessite de choisir la technologie adaptée à son implémentation et de définir un aperçu des interfaces de l'application. Ce chapitre présente la moyenne technologique utilisées pour le développement du projet, avec des captures d'écran illustrant ses fonctionnalités.

I. Modèle MVT (Model, View, Template)

Le modèle MVT (Modèle Vue-Template) est un modèle de conception logicielle qui divise une application en trois composants principaux : le Modèle, la Vue et le Template.

- ❖ **Modèle (Model) :** Représente les données et la logique métier de l'application, gère les opérations sur les données et interagit avec la base de données.
- ❖ **Vue (View) :** Représente l'interface utilisateur et la manière dont les données sont affichées, souvent sous forme de pages web ou d'interfaces graphiques.
- ❖ **Template :** Définit la structure de la vue et permet de présenter dynamiquement les données, en intégrant des variables ou balises dans les pages ou interfaces.

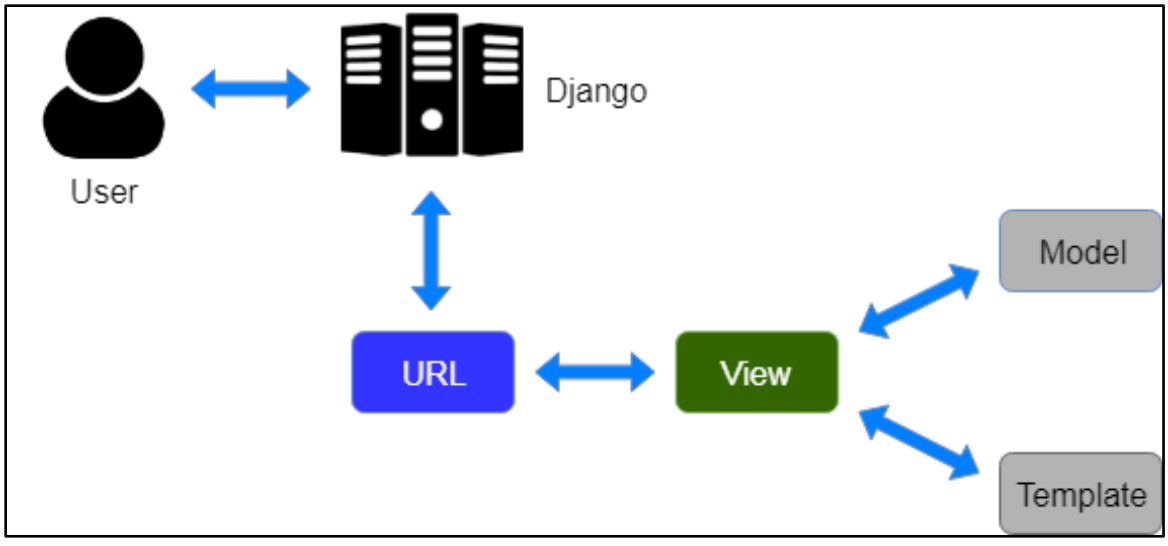


Figure 11 : Diagramme de modèle MVT

II. Interfaces

II.1. Interface Client

La figure ci-dessous représente la page d'accueil, qui constitue l'interface principale de notre application. À partir de cette page, le client peut accéder à son compte ou en créer un s'il n'en possède pas.

Cette interface peut être consultée par tout le monde sans besoin de s'authentifier.

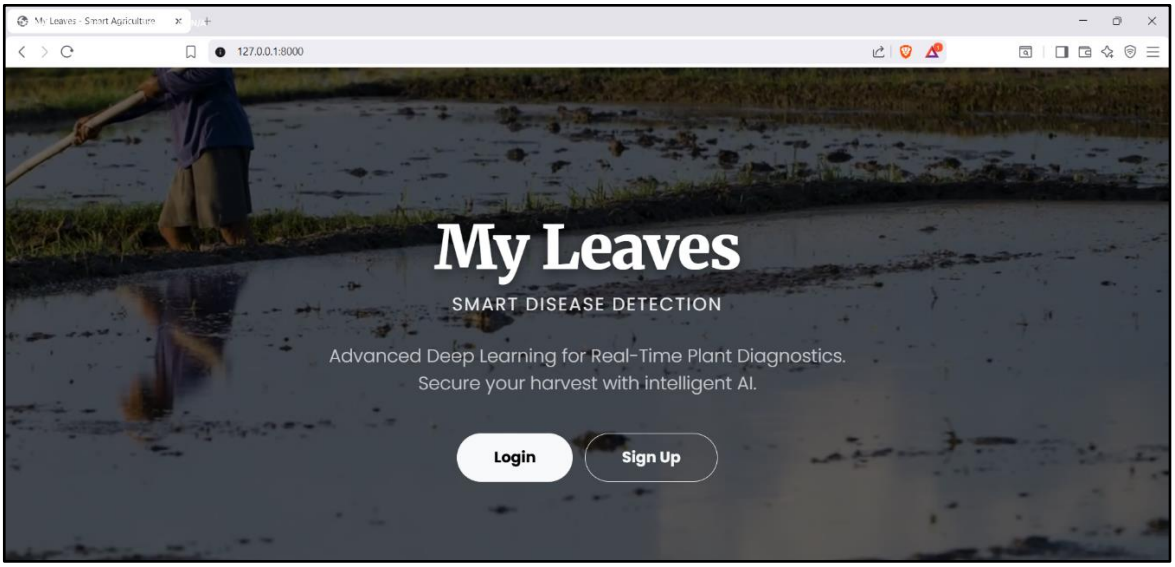


Figure 12 : Interface "Page d'accueil"

Pour effectuer une opération, l'utilisateur doit d'abord s'authentifier. S'il possède déjà un compte, il doit se connecter afin d'accéder à son espace. Dans le cas contraire, il doit créer un compte au préalable.

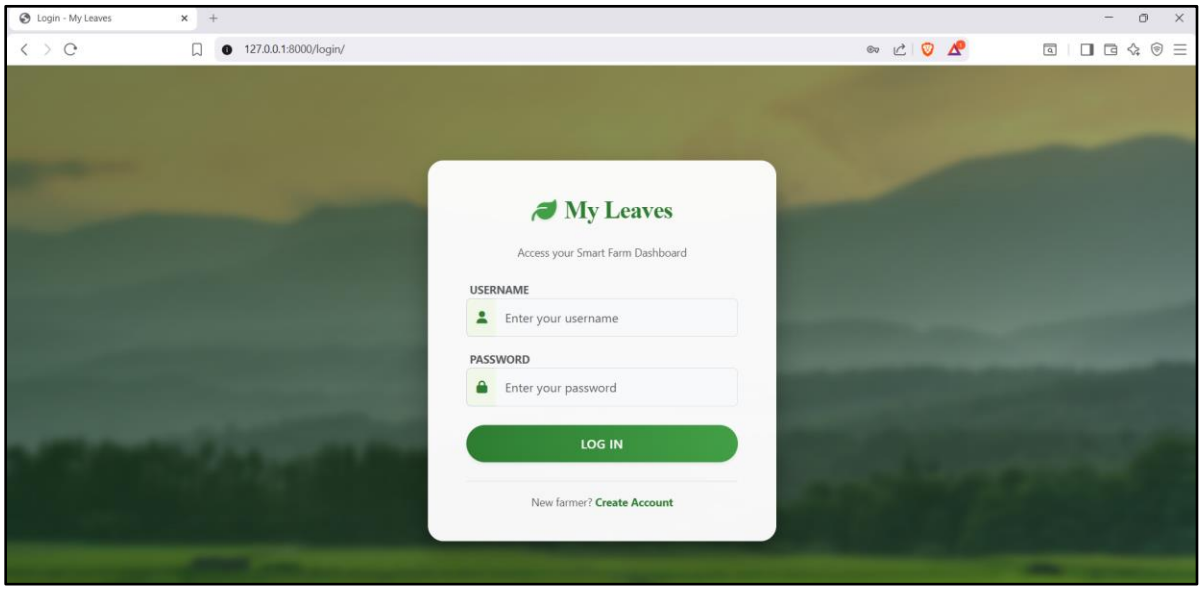


Figure 13 : Interface "Login" Pour les clients

Si le client ne possède pas encore de compte, il peut en créer un nouveau en cliquant sur "Sign up", où il devra remplir l'ensemble des champs nécessaires à la création de son compte.

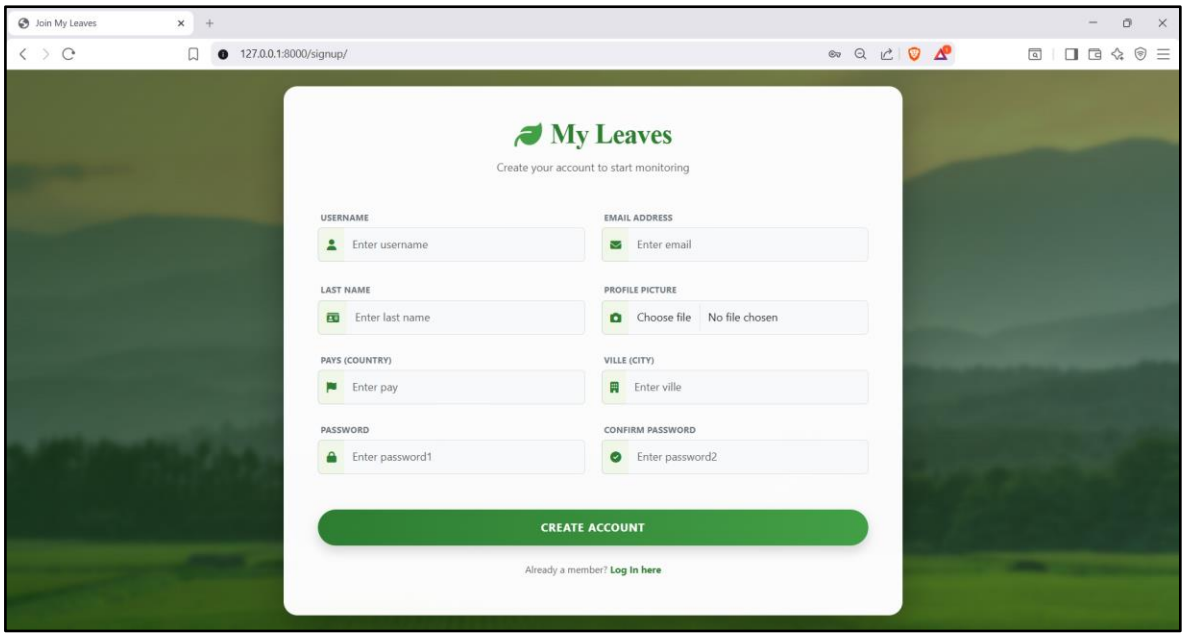


Figure 14 : Interface "Créer un copmte" Pour les clients

Après l'authentification de client, celui-ci accède à son espace. Cette interface affiche une zone permettant l'upload d'une image afin de réaliser son analyse, ainsi que des informations telles que le nombre total de scans déjà effectués et une autre zone permettant d'accéder à l'historique de toutes les analyses.

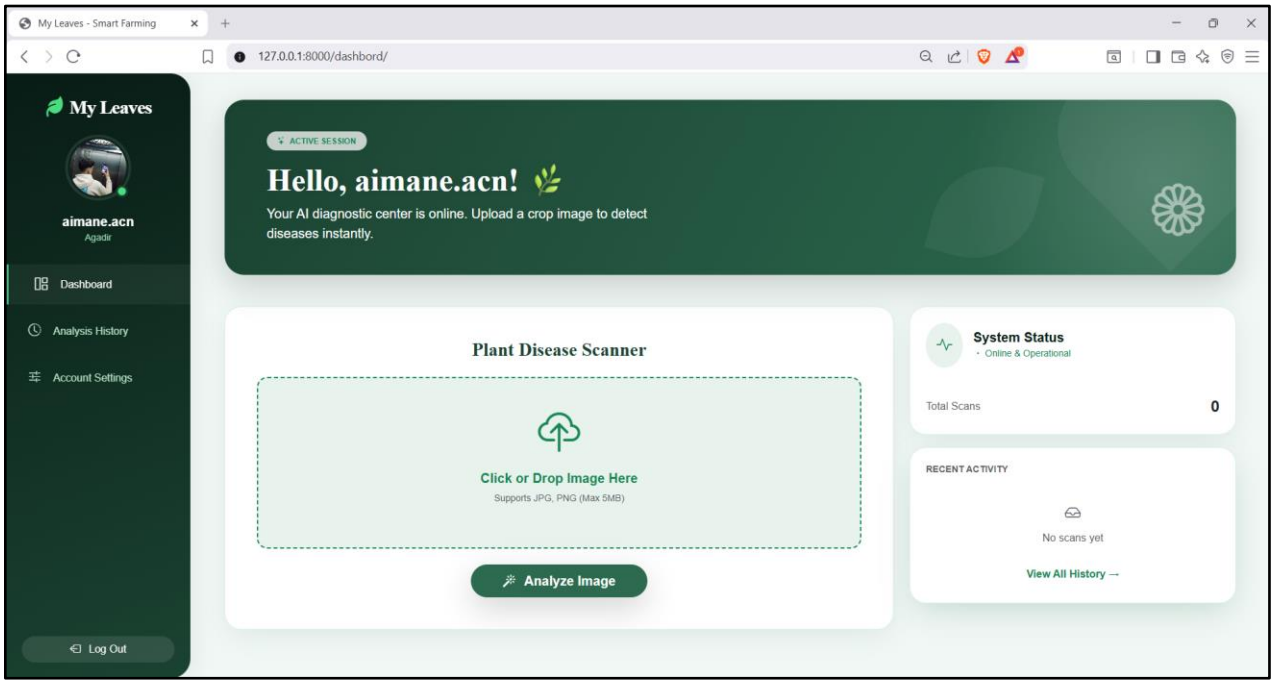


Figure 15 : Interface "Dashboard"

Pour effectuer une analyse, il suffit d'uploader l'image de la plante dans la zone spécifiée pour l'upload des images, puis de cliquer sur "Analyze image". Les résultats seront ensuite affichés.

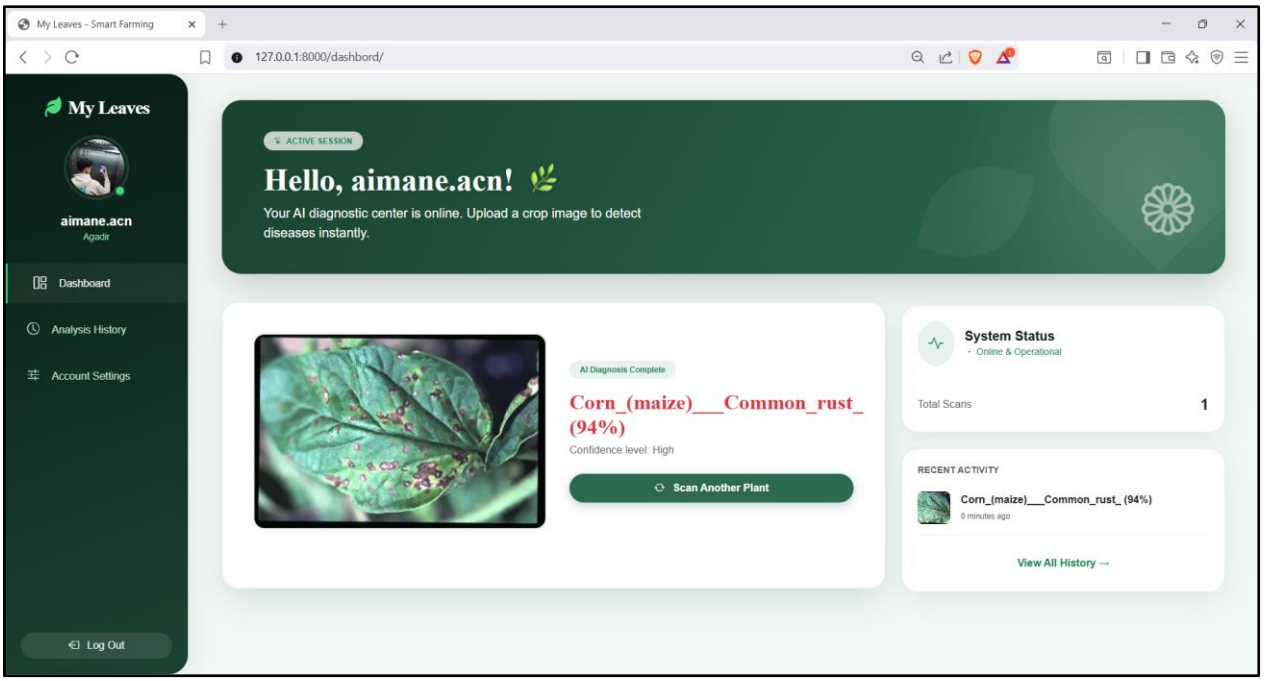


Figure 16 : Interface "Résultats d'une opération"

Pour consulter l'historique de toutes les analyses déjà effectuées avec leurs dates, il suffit de cliquer sur "View all History" ou sur "Analysis History" dans le menu, où il est également possible de supprimer un historique ou d'exporter l'historique au format PDF en cliquant sur "Print Report".

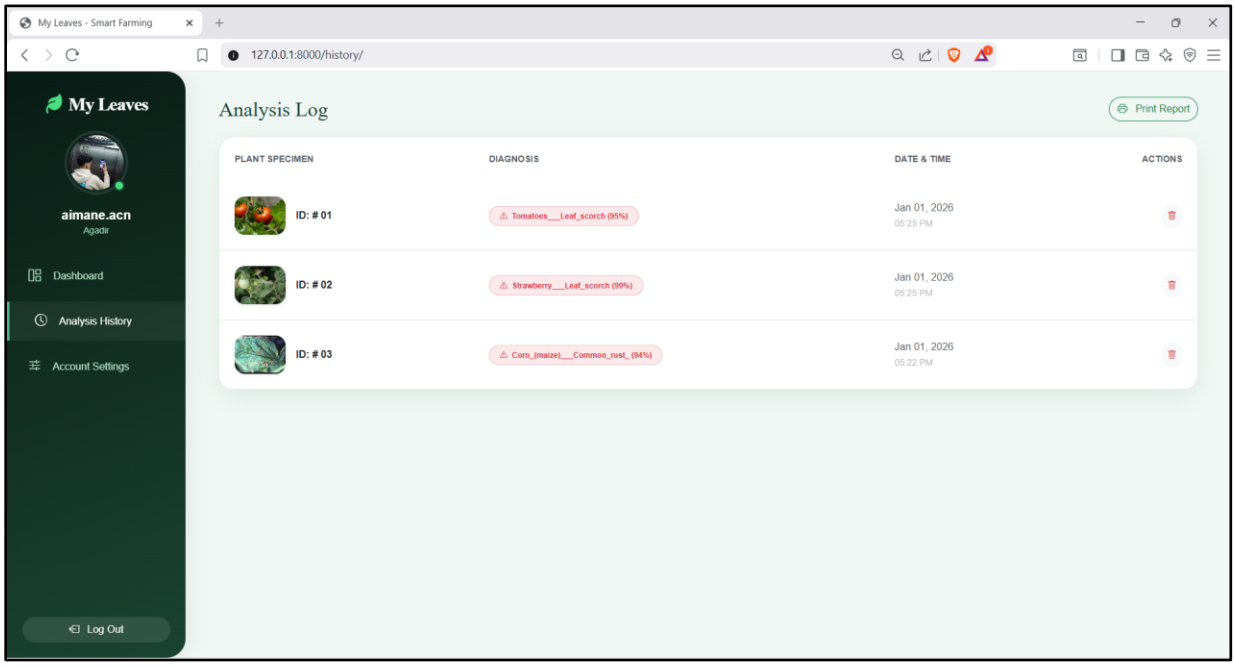


Figure 17 : Interface "Consulter l'historique"

II.2. Espace Administrateur

La figure ci-dessous représente l'interface d'authentification de l'application destinée à l'administrateur, à travers laquelle il se connecte à l'aide d'un identifiant et d'un mot de passe déjà enregistrés dans la base de données.

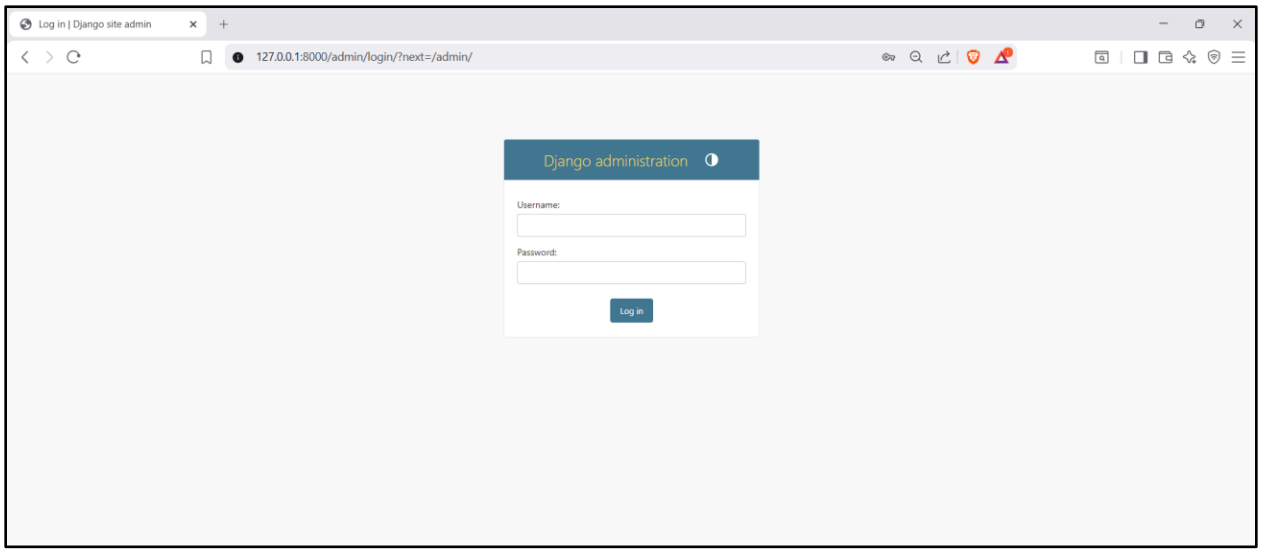


Figure 18 : Interface "Login" Pour l'admin

Après l'authentification, l'administrateur peut consulter l'ensemble des comptes des clients, où il a la possibilité de modifier les informations ou de supprimer un compte.

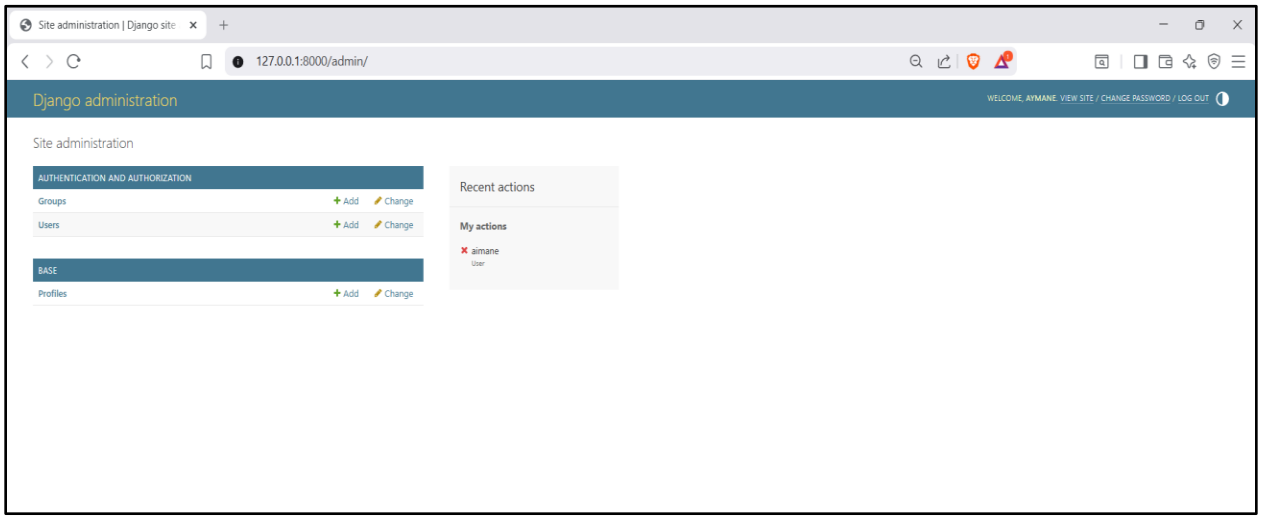


Figure 19 : Interface "Dashboard" Pour l'admin

Conclusion et perspective

En conclusion, on doit avouer que rétrospectivement nous sommes satisfaits de notre projet puisque nous avons atteint des nouveaux objectifs.

En effet, ce projet nous a permet de comprendre et apprendre à maîtriser plusieurs choses que nous ignorions avant, à l'aide de nos recherches et les conseils, les recommandations et les instructions de notre encadrant que nous tenons à remercie.

Nul ne peut nier qu'on a rencontré pas mal de problèmes, dans la plupart des cas on a pu trouver des solutions ensemble afin de les résoudre partiellement.

Le fait de travailler en groupe et d'utiliser des nouveaux outils, nous a permet de nous intégrer dans un groupe de travail et de voir en quoi consistait le travail sur une application web.

Enfin nous ne prétendons pas avoir réalisé le projet d'une manière parfaite, mais, nous sommes par ailleurs convaincus que le travail élaboré n'est qu'une étape primaire aussi bien pour une carrière professionnelle que pour des études plus approfondies.

Cependant des perspectives d'améliorations de notre application restent envisageables pour être enrichie par des fonctionnalités avancées telles que :

- Amélioration de côté du sécurité ;
- Ajouter plus des statistiques ;
- Ajouter une application mobile.

Webliographie

https://cdnjs.com/libraries/font-awesome

https://fontawesome.com/icons

https://docs.djangoproject.com/en/5.0/topics/forms/

https://www.geeksforgeeks.org/csrf-token-in-django/

https://docs.djangoproject.com/en/5.0/topics/http/views/

https://getbootstrap.com/

https://docs.djangoproject.com/en/5.0/topics/templates/

https://stackoverflow.com/questions

https://www.python.org/doc/

https://www.chartjs.org/docs/latest/

https://www.mailinblack.com/

https://grpc.io/docs/

https://developers.google.com/protocol-buffers

https://www.tensorflow.org/tutorials/images/cnn

https://http2.github.io/

https://www.sqlite.org/docs.html

https://cloud.google.com/storage